

■ Conversational Memory & Context Feature

Overview

The Property RAG System now includes **Conversational Memory & Context**, enabling multi-turn conversations where the system remembers previous queries and understands contextual follow-up questions.

■ Key Features

1. Session Management

Each conversation gets a unique session ID Sessions persist across queries Automatic session timeout (60 minutes of inactivity) Create new conversations or clear existing ones

2. Context Awareness

System remembers previous queries in a conversation Understands follow-up questions like: "What about cheaper ones?" "Show me those in London instead" "How about 2 bedrooms?" Automatically applies context from previous interactions

3. Smart Query Enhancement

Detects follow-up questions using indicators Enhances queries with context from conversation history
Merges filters from previous queries when appropriate

4. Conversation History

Stores full conversation history View past messages in the chat Persistent storage to disk Analytics on conversation patterns

■ Architecture

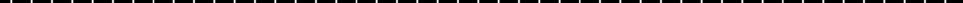
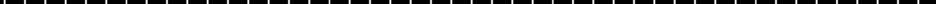
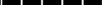
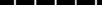
Components

Streamlit Frontend ■ ■ - Session UI ■ ■ - Conversation History Display ■

Backend ■ ■ - Session-aware endpoints ■ ■ - Conversation API ■

Conversation Manager ■ ■ - Session tracking ■ ■ - Message storage ■ ■ - Context extraction ■

Enhanced RAG Pipeline ■ ■ - Context-aware query enhancement ■ ■ - Filter inheritance ■ ■ - Smart

retrieval 
 
 Enhanced LLM Handler  - Conversation-aware prompts  - History integration  - Contextual responses 


backend/conversation_manager.py

Key Classes

Message structure

Session management

■ API Endpoints

1. POST /query (Enhanced)

```
json { "query": "What about cheaper ones?", "n_results": 5, "session_id": "uuid-here" }
```

Response includes: `json { "answer": "...", "properties": [...], "session_id": "uuid-here", "has conversation context": true }`

Create a new conversation session:

```
bash curl -X POST http://localhost:8000/conversation/new
```

```
Response: json { "session_id": "abc-123-def", "message": "New conversation session created" }
```

3. GET /conversation/{session_id}/history

Get conversation history:

```
bash curl http://localhost:8000/conversation/{session_id}/history?limit=10
```

```
Response: json { "session_id": "abc-123-def", "messages": [ { "role": "user", "content": "Show me 2  
bedroom apartments" }, { "role": "assistant", "content": "Found 5 properties..." } ], "count": 2 }
```

4. DELETE /conversation/{session_id}

Clear a conversation:

```
bash curl -X DELETE http://localhost:8000/conversation/{session_id}
```

5. GET /conversation/stats

Get statistics about all conversations:

```
bash curl http://localhost:8000/conversation/stats
```

```
Response: json { "active_sessions": 5, "total_messages": 127, "total_queries": 64,  
"avg_messages_per_session": 25.4 }
```

6. GET /conversation/{session_id}/context

Get contextual information for a session:

```
bash curl http://localhost:8000/conversation/{session_id}/context
```

■ Frontend Features

New UI Elements

Conversation Controls ■ New Chat - Start fresh conversation ■■ Clear - Clear current conversation

Session status indicator

Conversation History

Expandable history viewer Shows all previous messages

Timestamps and metadata

Context Indicators

Visual feedback when using context "■ Using conversation context" message

Session State Management

```
python st.session_state['session_id'] # Current session ID st.session_state['conversation_history'] # Local history cache st.session_state['query_count'] # Message counter
```

■ How It Works

Example Conversation Flow

User Query 1: "Show me 2 bedroom apartments under £1500"

System: - Creates new session (if none exists) - Searches for 2BR apartments under £1500 - Stores query + response in session - Returns results with session_id

User Query 2: "What about ones in Manchester?"

System: - Retrieves session context - Detects follow-up question (mentions "what about") - Extracts previous filters: bedrooms=2, max_price=1500 - Adds location filter: Manchester - Enhanced query: "apartments Manchester 2 bedroom" - Searches with combined filters - Returns contextual response

User Query 3: "Show me cheaper ones"

System: - Recognizes comparative request - Maintains previous filters and location - Adjusts price range downward - Provides refined results

Context Extraction Logic

The system automatically extracts:

Previous Filters Price ranges Bedroom/bathroom counts

Property types

Mentioned Locations

Cities, areas, neighborhoods

Extracted from query text

Property Types

Apartments, houses, studios, etc.

Used for query enhancement

User Preferences

Frequently used filters Common query patterns

Follow-up Detection

System recognizes follow-up questions through indicators: - "what about", "how about", "show me" - "those", "these", "same", "similar" - "cheaper", "expensive", "bigger", "smaller" - "also", "instead", "but", "other", "more"

■ Data Storage

Session Persistence

Sessions are stored in: backend/analytics/conversations/

Each session saved as: {session_id}.json

```
json { "session_id": "abc-123-def", "messages": [ { "role": "user", "content": "...", "metadata": {...},  
"timestamp": "2025-10-08T10:30:00" } ], "created_at": "2025-10-08T10:25:00", "last_accessed":  
"2025-10-08T10:35:00", "metadata": { "query_count": 3, "total_properties_viewed": 15, "filters_used":  
["bedrooms", "max_price"], "topics": ["apartments", "manchester"] } }
```

Session Cleanup

Timeout: 60 minutes of inactivity **Automatic cleanup:** Removes expired sessions **Max sessions:** 1000 active sessions (configurable)

■ Usage Examples

Basic Flow

Start conversation: python # First query creates session automatically POST /query { "query": "Find 3 bedroom houses" } # Returns: session_id

Continue conversation: python POST /query { "query": "What about ones under £2000?", "session_id": "previous-session-id" } # Uses context from first query

View history: python GET /conversation/{session_id}/history

Advanced Usage

```
```python import requests
```

## Create session

```
response = requests.post("http://localhost:8000/conversation/new") session_id =
response.json()["session_id"]
```

## Query 1

```
query1 = { "query": "Show me apartments in London", "session_id": session_id, "n_results": 5 } result1 =
requests.post("http://localhost:8000/query", json=query1)
```

## Query 2 (follow-up)

```
query2 = { "query": "What about 2 bedroom ones?", "session_id": session_id } result2 =
requests.post("http://localhost:8000/query", json=query2)
```

## Automatically applies London filter from Query 1

## Get full conversation

```
history = requests.get(f"http://localhost:8000/conversation/{session_id}/history") print(history.json()) ```
```

## ■ Configuration

### Conversation Manager Settings

```
```python
```

In backend/conversation_manager.py

```
ConversationManager( persist_directory="analytics/conversations", # Storage location  
session_timeout_minutes=60, # Session expiry time max_sessions=1000 # Maximum active sessions ) ```
```

Session Settings

```
```python
```

## Per-session settings

```
ConversationSession(session_id="...", max_history=10 # Keep last 10 message pairs) ```
```

## ■ Testing

### Test Script

```
```python
```

Test conversation memory

```
from backend.conversation_manager import ConversationManager  
manager = ConversationManager()
```

Create session

```
session_id = manager.create_session()
```

Add messages

```
manager.add_message_to_session( session_id, 'user', 'Show me 2 bedroom apartments', {'filters':  
{ 'bedrooms': 2 }} )  
  
manager.add_message_to_session( session_id, 'assistant', 'Found 5 properties...' )
```

Get context

```
context = manager.get_session_context(session_id) print(context)
```

Get history

```
history = manager.get_conversation_history(session_id) print(history) ``
```

■ Benefits

Natural Conversations Users can ask follow-up questions naturally

No need to repeat context every time

Better User Experience

Feels like chatting with a person

Faster queries (less typing)

Improved Results

Context helps refine searches

Better understanding of user intent

Analytics

Track conversation patterns Understand user behavior across sessions

■ Future Enhancements

Potential improvements:

Long-term Memory Remember user preferences across sessions

Personalization based on history

Multi-user Support

User authentication

Private conversation histories

Advanced Context

Entity recognition Sentiment analysis

Intent classification

Conversation Analytics

Common conversation flows Drop-off analysis

Topic clustering

Export Conversations

Download chat history Share conversations

■ Related Documentation

Main README API Documentation Setup Guide

■ Troubleshooting

Session Not Persisting

Check backend/analytics/conversations/ directory exists Verify write permissions

Context Not Working

Ensure session_id is passed in requests Check session hasn't expired (60 min timeout) Verify backend logs for context extraction

High Memory Usage

Reduce max_sessions setting Lower session_timeout_minutes Reduce max_history per session

■ Support

For issues or questions about conversational memory: - Check backend logs: Look for "ConversationManager" messages - Verify session existence: GET /conversation/{session_id}/context - Test conversation flow: Use provided test scripts

Version: 2.0.0

Last Updated: October 8, 2025

Feature Status: ■ Production Ready